Числа с плавающей точкой

внутреннее устройство чисел с плавающей точкой, машинное эпсилон, кумулятивная ошибка, сумма с компенсацией, суммирование Кехена, fused multiply add (fma)

Общие сведения о числах с плавающей точкой

Binary32 IEEE-754 / float



0	0 1 1 1 1 1 1 1	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0	1 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
1	1 0 0 0 0 0 1 0	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0	1 0 0 0 0 0 0 0	1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

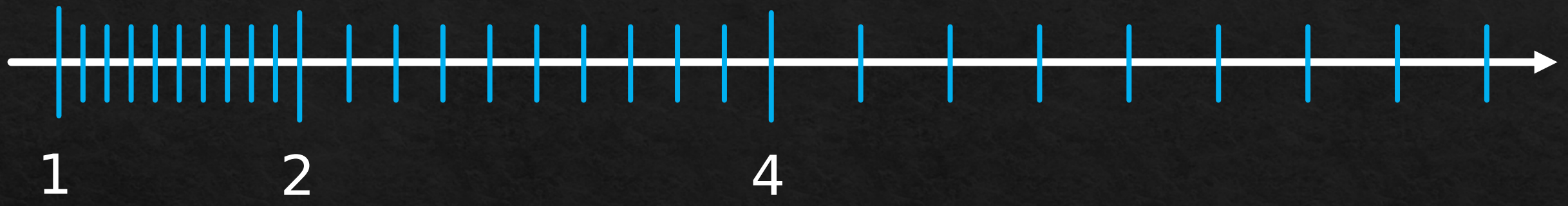
1

2

-8

3

Числа с плавающей точкой не являются всюду плотными

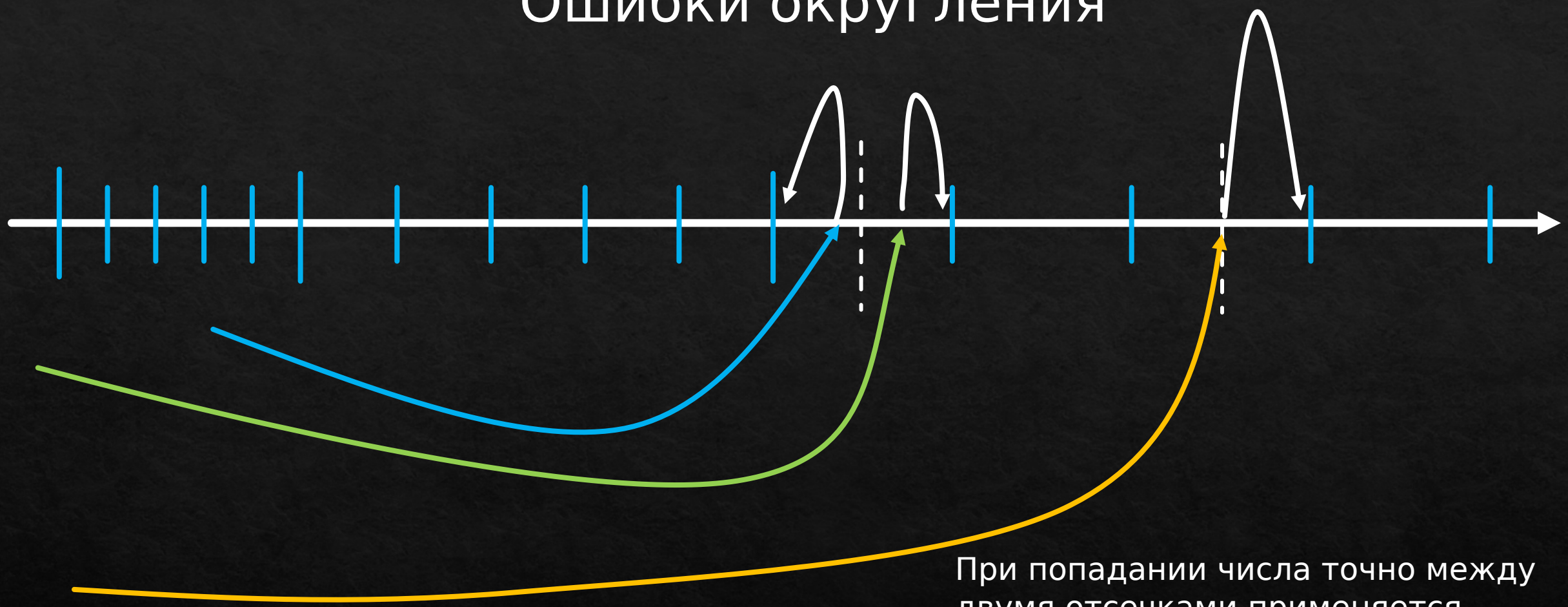


Float – это набор отсечек на числовой прямой. Причём они устроены таким образом, что при увеличении числа вдвое промежутки между соседними числами тоже увеличиваются в два раза. Хотя абсолютная погрешность увеличивается, относительная погрешность сохраняется.

$x_0 + \Delta x$ абсолютная погрешность

$\frac{\Delta x}{x_0}$ относительная погрешность

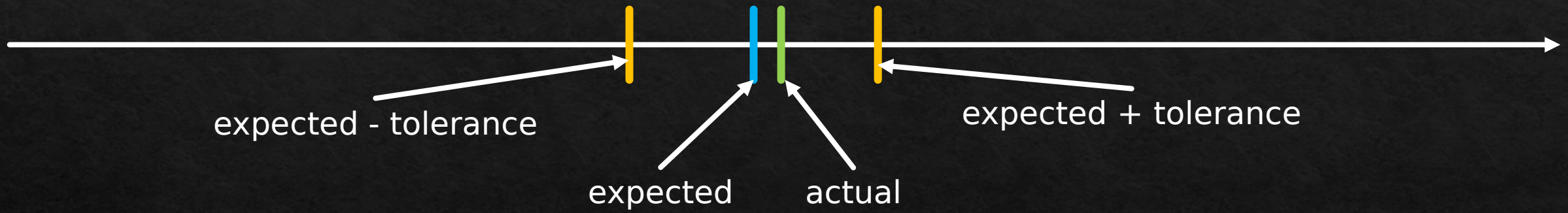
Ошибки округления



При попадании числа точно между двумя отсечками применяется правило Кехена: округление идёт в сторону ближайшей чётной по счёту отсечки.

Сравнение чисел с плавающей точкой

Из-за ошибок округления невозможно сравнивать числа с плавающей точкой на точное равенство. Следует ввести некоторый допуск (tolerance).




```
17  int main() {
18      float dx = 0.01f;
19      float accum = 0.f;
20      float tolerance = 2.e-4f;
21      for (unsigned cnt = 0; cnt != 1024; ++cnt)
22          accum += dx;
23
24      cout << boolalpha;
25      cout << (accum == dx * 1024.f) << endl;
26      cout << (accum + tolerance > dx * 1024.f && dx * 1024.f > accum - tolerance) << endl;
27
28      return 0;
29  }
```

```
false
true
```

Номер отсечки

```cpp
#include <iostream>
#include <iomanip>

using namespace std;

int main() {
    cout << setprecision(9) << fixed;

    union fused_float_unsigned_t {
        float f;
        unsigned u;
    } fu;

    fu.f = 1.f;
    cout << fu.f << " " << fu.u << '\n';

    fu.u += 1;
    cout << fu.f << " " << fu.u << '\n';
}
```

```
1.000000000 1065353216
1.000000119 1065353217
```

Вывод чисел с плавающей точкой

```cpp
#include <iostream>
#include <iomanip> // setprecision

using namespace std;

int main() {
    float pi = 3.14159265f;
    float unit = 1.f;

    cout << unit << " " << pi << '\n';

    cout << setprecision(4) << fixed;
    cout << unit << " " << pi << '\n';

    cout << setprecision(4) << scientific;
    cout << unit << " " << pi << '\n';

    cout << endl;

    return 0;
}
```

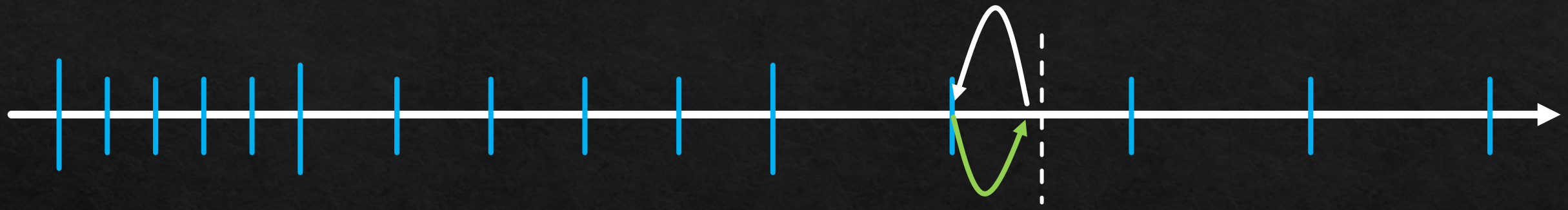
```
1 3.14159
```

```
1.0000 3.1416
```

```
1.0000e+00 3.1416e+00
```

Машинное эпсилон

Из-за конечного расстояния между числами с плавающей точкой возможна ситуация, когда к некоторому числу добавляется настолько малое значение, что округление происходит назад к самому числу.



```
17  int main() {
18      float const unit = 1.f;
19      float epsilon = 1.f;
20
21      while (unit + epsilon != unit) epsilon /= 2;
22      cout << epsilon << endl;
23
24      return 0;
25  }
```

```
5.96046e-08
```

Минимальное значение, которое можно добавить к единице и получится значение большее единицы (мы переходим через середину между единицей и следующей отсечкой) называется «машинное эпсилон». Его значение, разумеется, зависит от разрядности числа. Точное значение можно получить опираясь на данные std::numeric limits<type>::epsilon().

```
17  #include <limits>
18
19  int main() {
20      cout << numeric_limits<float>::epsilon() << '\n';
21      cout << numeric_limits<double>::epsilon() << '\n';
22      cout << endl;
23
24      return 0;
25  }
```

```
1.19209e-07
2.22045e-16
```

Кумулятивная ошибка

При вычислении суммы рядов может возникнуть такая ситуация, что следующий элемент ряда настолько мал, что его невозможно прибавить к полной сумме, значение оказывается меньше машинного эпсилон (с учётом роста погрешности).

```
 7  void harmonic_series(float &result, float &term) {
 8      float prev_sum = 0.f, curr_sum = 1.f;
 9      unsigned N = 2;
10      while(curr_sum != prev_sum) {
11          cout << curr_sum << " " << 1.f/N << " " << N << '\n';
12          prev_sum = curr_sum;
13          curr_sum += 1.f/N;
14          ++N;
15      }
16      result = curr_sum;
17      term = 1.f/N;
18  }
```

```
1.540368e+01 4.768372e-07 2097152
```

Как правильно?

Равносильные преобразования

$$\sqrt{x+\alpha}-\sqrt{x-\alpha}$$

В таком виде мы можем получить строгий ноль, если $\frac{\alpha}{x}$ мало.

$$\frac{2\alpha}{\sqrt{x+\alpha}+\sqrt{x-\alpha}}$$

Выполнив алгебраическое преобразование, мы избавились от одинаково больших частей наших выражений и оставили только значимую малую часть.

$$\sqrt{x}\left(\sqrt{1+\frac{\alpha}{x}}-\sqrt{1-\frac{\alpha}{x}}\right)\approx\sqrt{x}\left(\frac{\alpha}{2x}+\frac{\alpha}{2x}\right)=\frac{\alpha}{\sqrt{x}}$$

Error free transformations (EFT)

$$a + b = (a \oplus b) + t$$

$+$ – точная математическая сумма чисел, представимых в типе данных

$\oplus$ – операция сложения двух чисел с плавающей точкой в рамках машинной арифметики

t – некоторое представимое в типе число

```
3  void TwoSum(float const a, float const b, float &s, float &t) {
4      s = a + b;
5      auto z = s - a;
6      t = (a - (s - z)) + (b - z);
7  }
```

the Veltkamp’s / Veltkamp-Dekker algorithm

$a = s + t$

Точное разбиение числа на более старшие и более младшие разряды.

```
 9  void Split(float const x, float &x_h, float &x_l) {
10      auto c = static_cast<float>((1ul << 11) + 1ul);
11      x_h = (c * x) + (x - (c * x));
12      x_l = x - x_h;
13  }
```

$$a \times b = (a \otimes b) + t$$

$\times$ – точное перемножение чисел, представимых в типе данных

$\otimes$ – операция перемножения двух чисел с плавающей точкой в рамках машинной арифметики

```
15  void TwoMult(float const a, float const b, float &s, float &t) {
16      double a_high, a_low, b_high, b_low;
17      Split(a, a_high, a_low);
18      Split(b, b_high, b_low);
19      s = a * b;
20      t = -s + a_high * b_high;
21      t += a_high * b_low;
22      t += a_low * b_high;
23      t += a_low * b_low;
24  }
```

Суммирование рядов

Последовательность суммирования

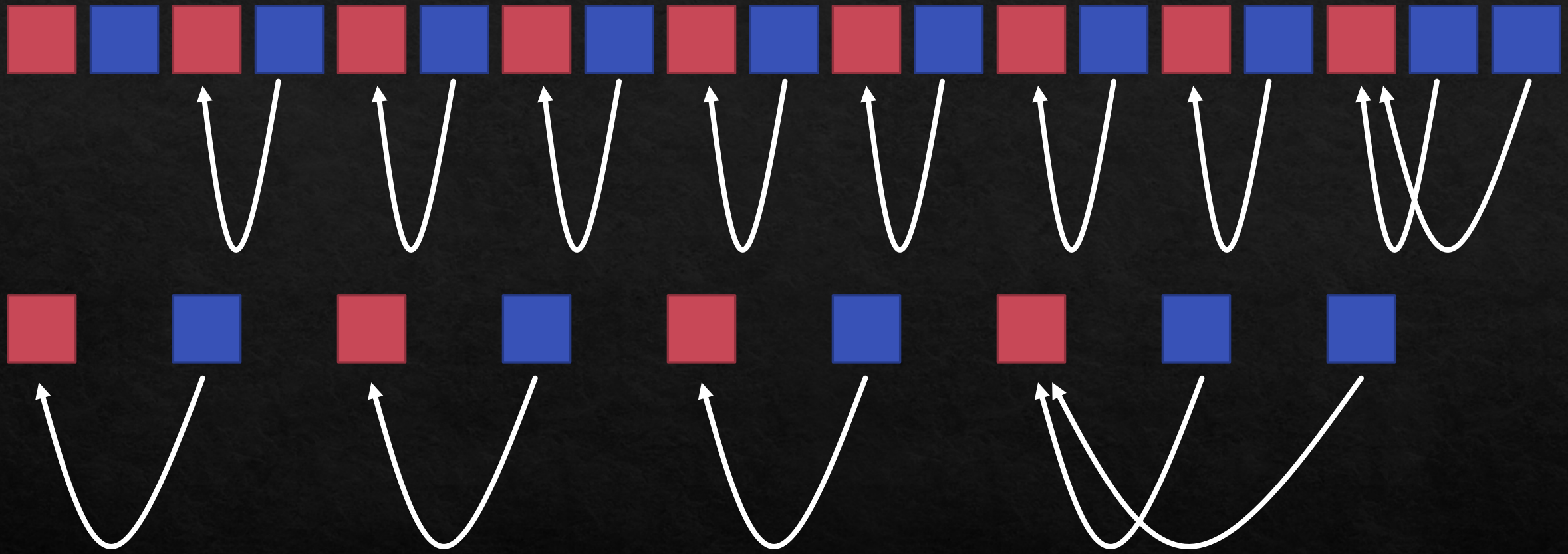
```
27  float forward_sum_harmonic_series(unsigned max_N) {
28      float sum = 0.f;
29      for (unsigned N = 1; N != max_N + 1; ++N)
30          sum += 1.f/N;
31      return sum;
32  }
```

```
20  float backward_sum_harmonic_series(unsigned max_N) {
21      float sum = 0.f;
22      for (unsigned N = max_N; N != 0; --N)
23          sum += 1.f/N;
24      return sum;
25  }
```

```
34  int main() {
35      cout << setprecision(6) << scientific;
36      cout << backward_sum_harmonic_series(5'000'000u) << endl;
37      cout << forward_sum_harmonic_series(5'000'000u) << endl;
38
39      return 0;
40  }
```

```
1.600785e+01
1.540368e+01
```

Суммирование близких и суммирование пополам



Суммирование Кехена

Алгоритм суммирования Кехена – это алгоритм суммирования рядов на основе EFT.

```
34  float forward_kahan_sum_harmonic_series(unsigned max_N) {
35      float sum = 0.f;
36      float t = 0.f;
37      for(unsigned N = 1; N != max_N + 1; ++N) {
38          float y = 1.f/N - t;
39          float z = sum + y;
40          t = (z - sum) - y;
41          sum = z;
42      }
43      return sum;
44  }
```

```
cout << forward_sum_harmonic_series(5'000'000u) << endl;
cout << backward_sum_harmonic_series(5'000'000u) << endl;
cout << forward_kahan_sum_harmonic_series(5'000'000u) << endl;
cout << backward_kahan_sum_harmonic_series(5'000'000u) << endl;
```

```
1.540368e+01
1.600785e+01
1.600216e+01
1.600216e+01
```

Заповеди работы с числами с плавающей точкой

Не работай с числами с плавающей точкой!

Правильно нормируй задачу

Выбирай самый «большой» доступный тип

Используй специальные приёмы и возможности